1. Time Complexity of asc_partition

This function partitions the array around a pivot for quicksort. Here's the line-by-line analysis:

```
int asc_partition(int A[], int l, int h) {
   int pivot = A[l];          // O(1)
   int i = l; int j = h;      // O(1)
   while (i < j) {            // O(n) worst case
      while (A[i] <= pivot) { // O(n) worst case
         i++;                 // O(1)
      }
      while (A[j] > pivot) {  // O(n) worst case
         j--;                 // O(1)
      }
      if (i < j) {            // O(1)
         swap(A[i], A[j]);    // O(1)
      }
   }
   swap(A[l], A[j]);          // O(1)
   return j;                  // O(1)
}
```

- int pivot = A[l]; → **O(1)**
    - Assigning a single value is constant time.
- int i = l; int j = h; → **O(1)**
    - Two assignments, each constant time.
- while (i < j) { → **O(n) worst case**
    - This outer loop runs until i and j meet. In the worst case (e.g., sorted or reverse sorted array), it processes all elements between l and h, where n = h - l + 1. The total work depends on the inner loops.
- while (A[i] <= pivot) { i++; } → **O(n) worst case**
    - In the worst case (e.g., all elements ≤ pivot), this loop iterates from l to h, taking O(n) time. Each iteration is O(1).
- while (A[j] > pivot) { j--; } → **O(n) worst case**
    - In the worst case (e.g., all elements > pivot), this loop iterates from h to l, taking O(n) time. Each iteration is O(1).
- if (i < j) { swap(A[i], A[j]); } → **O(1)**
    - The condition check is O(1), and swap (assuming it's a simple exchange) is O(1). This happens at most once per outer loop iteration.
- swap(A[l], A[j]); → **O(1)**
    - A single swap operation outside the loop, constant time.
- return j; → **O(1)**
    - Returning a value is constant time.

Total Time Complexity of asc_partition:

- The outer while (i < j) loop, combined with the inner while loops, processes each element at most once as i moves right and j moves left.
- In the worst case (e.g., already sorted or reverse sorted array), i and j traverse the entire subarray, leading to O(n) comparisons and O(1) swaps per partition.
- Thus, the total time complexity is **O(n)**, where n = h - l + 1.

2. Time Complexity of asc_quick_sort

This function implements the quicksort algorithm recursively. Here's the line-by-line analysis:

```
void asc_quick_sort(int A[], int l, int h) {
   if (l < h) {                  // O(1)
      int j = asc_partition(A, l, h);  // O(n)
      asc_quick_sort(A, l, j - 1);     // T(n1)
      asc_quick_sort(A, j + 1, h);     // T(n2)
   }
}
```

- if (l < h) { → **O(1)**
  - A simple comparison, constant time.
- int j = asc_partition(A, l, h); → **O(n)**
  - Calls asc_partition, which takes O(n) time as derived above, where n = h - l + 1.
- asc_quick_sort(A, l, j - 1); → **T(n1)**
  - Recursive call on the left subarray, size n1 = j - l. Time depends on the partition size, denoted as T(n1).
- asc_quick_sort(A, j + 1, h); → **T(n2)**
  - Recursive call on the right subarray, size n2 = h - j. Time depends on the partition size, denoted as T(n2).

Total Time Complexity of asc_quick_sort:

- The time complexity is defined by the recurrence relation:
  T(n) = T(n1) + T(n2) + O(n)
  where n1 + n2 = n - 1 (since the pivot j is excluded from both subarrays).
- The actual complexity depends on the pivot choice and input:
  - **Worst Case**: Pivot is always the minimum or maximum (e.g., sorted array).
    - Partition splits as 0 and n-1.
    - $T(n) = T(n-1) + O(n) \rightarrow O(n^2)$.
  - **Best Case**: Pivot is the median, splitting the array evenly.
    - $T(n) = 2 * T(n/2) + O(n) \rightarrow O(n \log n)$ (solved via Master Theorem).
  - **Average Case**: Random pivot, expected split is roughly balanced.
    - $T(n) \approx 2 * T(n/2) + O(n) \rightarrow O(n \log n)$ (via probabilistic analysis).

Final Answer

- **asc_partition Time Complexity**: **O(n)**
  - Linear time to partition the subarray of size n = h - l + 1.
- **asc_quick_sort Time Complexity**:
  - **Worst Case**: **$O(n^2)$** (unbalanced partitions).
  - **Best Case**: **O(n log n)** (balanced partitions).
  - **Average Case**: **O(n log n)** (randomized pivots).